

Log-Space Importance Model for Ethereum

Deep Funding GG24 · Level I · by Anas

Aura

Log-Space Importance Model for Ethereum · Deep Funding GG24 — Level I · by Anas

github.com/i-anasop/L3

98

REPOS WEIGHTED

48.6%

JURY VARIANCE EXPLAINED

log

NATIVE TARGET SPACE

L00

HONEST VALIDATION

Aura assigns relative importance weights to the 98 repositories Ethereum depends on. It is built around a single fact recovered from the contest's own scoring code: **the jury scores in log-space, on pairwise ratios**. Aura predicts log-weights, is trained in that exact geometry, and is validated against the real jury objective under leave-one-out cross-validation.

1 The scoring, exactly

The Deep Funding scoring mechanism (`deepfunding/scoring`) computes:

```
def cost_function(logits, samples):  
    return sum((logits[b] - logits[a] - c) ** 2 for a, b, c in samples)
```

Each jury sample `(a, b, c)` gives two repos and `c`, the log of how much more important `b` is than `a`. The cost is the squared error between your **predicted log-ratio** and the jury's observed log-ratio. Three consequences define the model:

- **The target is log-weight.** Aura predicts in log-space and trains to match pairwise log-ratios — the scoring's own geometry, not raw weights.
- **The score is scale-invariant.** It depends only on *differences* of log-weights, so overall normalization is irrelevant. Only the **shape** — relative spacing of repos in log-space — matters.
- **Spread is the master parameter.** Since only relative spacing counts, the dispersion of log-weights is the single highest-leverage quantity to calibrate.

2 The model

A regularized **log-space ridge regression** trained on the 50 repos with public jury weights, predicting all 98. Features were selected by forward search directly against the jury cost:

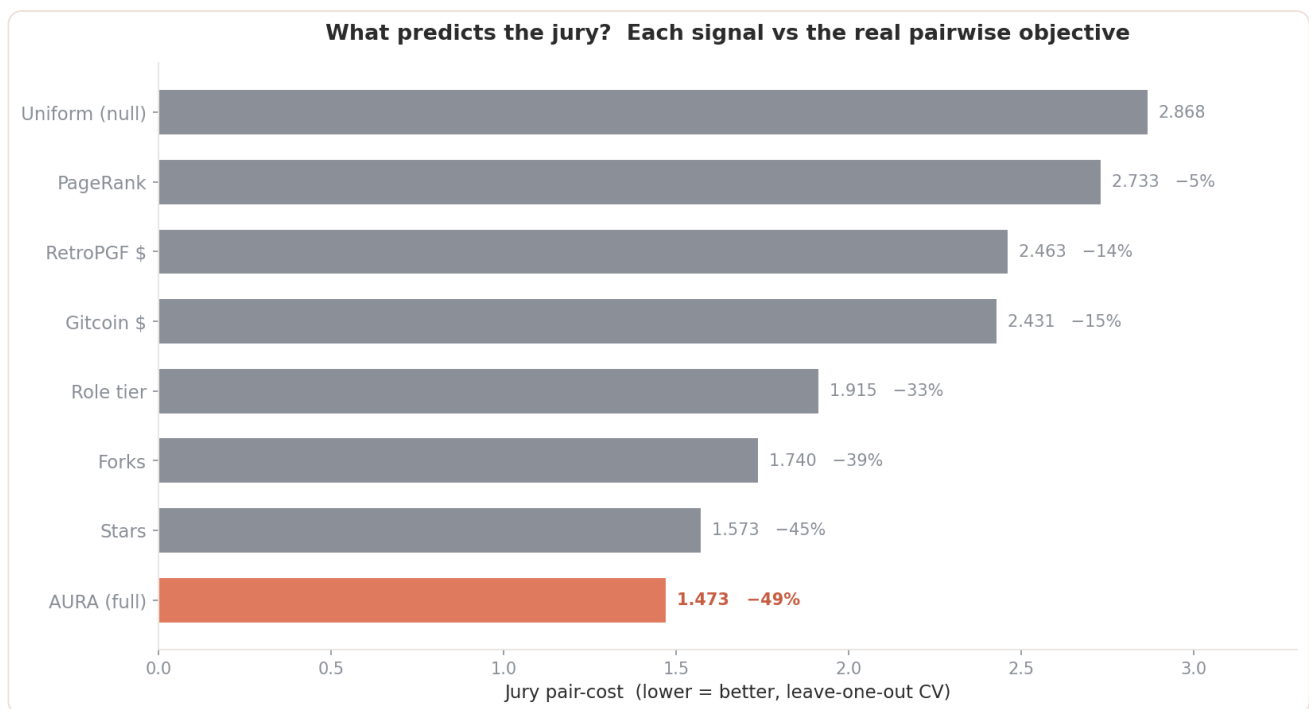
Feature	What it captures
<code>log_stars</code>	adoption — the dominant single predictor
<code>log_forks</code> , <code>log_size</code>	adoption breadth & code surface area

age_days	project maturity
tier_prior	ecosystem role (client / spec / library / tooling)
pagerank , has_graph	dependency-graph centrality
gitcoin_donors	community funding signal

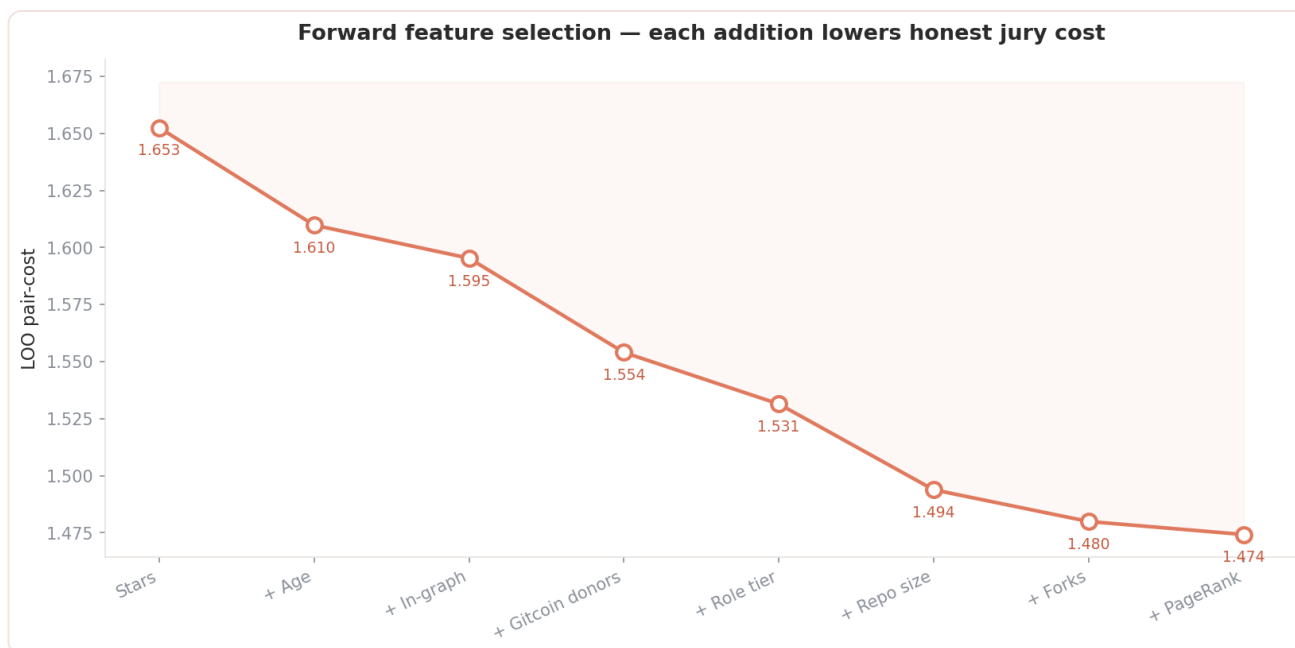
Pipeline: standardize → ridge ($\alpha=10$, heavy L2 for only 50 training points) → exponentiate → **calibrate the log-spread** to minimize held-out jury cost → normalize to sum 1.

3 Validation against the real jury

Every model is scored on the **171 public pairwise jury comparisons** with the exact cost function, under **leave-one-out CV** — the same generalization regime as the post-event held-out leaderboard.



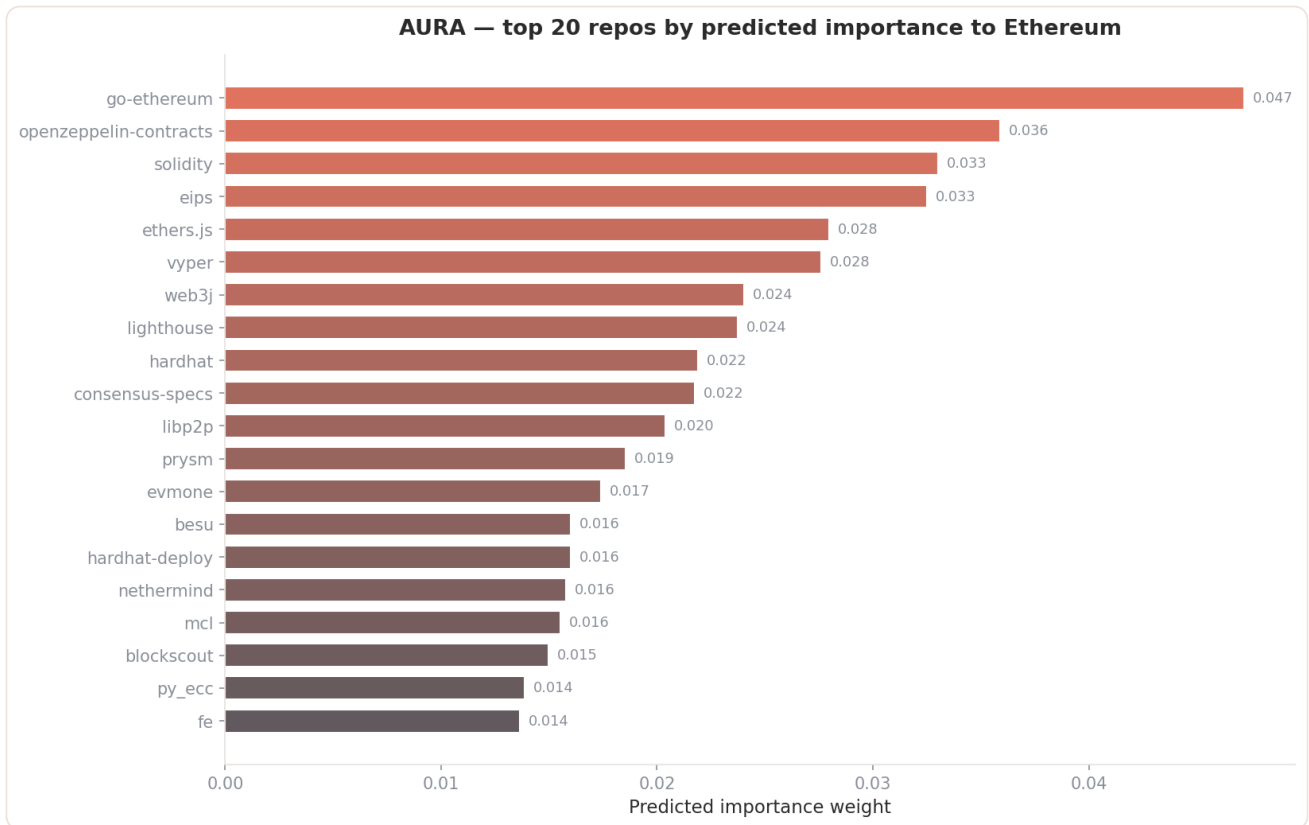
Model	Jury pair-cost	vs null
Uniform (null)	2.868	—
PageRank only	2.733	-5%
RetroPGF \$ only	2.463	-14%
Gitcoin \$ only	2.431	-15%
Role tier only	1.915	-33%
Forks only	1.740	-39%
Stars only	1.573	-45%
AURA (full)	1.473	-49%



4 Findings, stated honestly

- **Adoption dominates.** `log_stars` alone reaches -45%. The jury's notion of importance tracks real-world usage more than any other single feature.
- **Funding helps, modestly.** RetroPGF and Gitcoin dollars are positive predictors (-14/-15%) but sparse across the 98 repos and outranked by raw adoption.
- **Simpler generalizes better.** A gradient-boosted ensemble *lost* to plain ridge under LOO-CV (1.56 vs 1.47). With 50 training points, regularization and feature selection win — Aura ships the simpler model.
- **Spread calibration is real.** Because scoring is scale-invariant, tuning the log-weight dispersion directly lowers pair-cost; it is the highest-leverage single knob.

5 Predicted importance



6 Reproduce

```
pip install -r requirements.txt
cd src && python aura.py          # writes aura_submission.csv
python validate.py               # reproduces the L00 pair-cost table
```